

Medición de Tiempos de Interrupción

Guía de ensayos y análisis de resultados — FRDM-K64F

Santiago Riverós

Ignacio Sammartino

Rodrigo Devesa

Leandro Yiu

TP1 — Sistemas Embebidos

Resumen

Este documento define la campaña de mediciones con osciloscopio sobre el firmware del sistema de control de acceso, y contiene el análisis de los resultados. Cada ensayo indica qué se mide, cómo se configura el equipo y — sobre todo — qué conclusión de diseño permite extraer. El objetivo no es sólo caracterizar tiempos, sino *verificar experimentalmente* tres cosas: que el sistema cumple sus restricciones de tiempo real, cuánto cuestan las abstracciones de la arquitectura en capas, y que la asignación de prioridades de interrupción es correcta.

Índice

1. Método

1.1. El pin de medición

El firmware incluye un módulo de instrumentación (`source/test_isr_cfg.h`) que pone el pin `PIN_TEST_ISR` en alto al entrar a una rutina y en bajo al salir. De ese modo, en el osciloscopio:

- el **ancho del pulso** es el tiempo de ejecución de la rutina;
- el **período entre flancos de subida** es cada cuánto se ejecuta;
- el **duty** es el porcentaje de CPU que consume.

Sólo se puede medir una rutina por vez, porque hay un único pin físico. La selección se hace cambiando una línea y recompilando:

```
#define TEST_ISR_TARGET          TEST_ISR_DISPLAY_REFRESH
```

Listing 1: Selección de la rutina a medir en `test_isr_cfg.h`.

La instrumentación se resuelve en tiempo de compilación: los puntos de medición no seleccionados expanden a `((void)0)` y no generan ni una instrucción. El binario con `TEST_ISR_NONE` es idéntico en tamaño al que tiene la medición activa, de modo que medir no altera lo medido más allá de las dos escrituras al GPIO.

1.2. Condiciones del ensayo

Parámetro	Valor
Microcontrolador	MK64FN1M0VLL12 (Cortex-M4)
Frecuencia de core	100 MHz (1 ciclo = 10 ns)
Optimización	-O0 (build de Debug)
Frecuencia del SysTick	1 kHz (período 1 ms)
Pin de medición	<code>PIN_TEST_ISR = PTC17</code>
Osciloscopio	_____
Sonda / atenuación	_____

Cuadro 1: Condiciones bajo las que se realizaron todas las mediciones.

Importante: los tiempos corresponden a una compilación sin optimizar. Con `-O2` los valores absolutos bajarían de forma apreciable; las relaciones entre ellos, que son la base de las conclusiones, se mantienen.

1.3. Configuración del osciloscopio

1. Punta en PTC17, masa a cualquier GND de la placa.
2. Acoplamiento DC, disparo por **flanco de subida** en $\approx 1,5$ V.
3. Para medir el **ancho**: base de tiempo corta, de modo que el pulso ocupe buena parte de la pantalla. Usar la medición automática de *ancho de pulso positivo*.
4. Para medir el **período**: base de tiempo larga, hasta ver varios pulsos. Usar la medición automática de *período* o *frecuencia*.
5. Anotar siempre las dos cosas: con ancho y período se calcula el duty.

1.4. Cálculos a aplicar

Para cada rutina medida, con ancho t_{on} y período T :

Magnitud	Fórmula	Qué significa
Frecuencia de ejecución	$f = 1/T$	cada cuánto corre
Carga de CPU	$D = t_{on}/T \cdot 100 \%$	cuánta CPU consume
Ciclos de reloj	$N = t_{on} \cdot 100 \text{ MHz}$	costo en instrucciones

2. Ensayos

Los ensayos están ordenados a propósito: M1 establece la base de tiempo, M2–M4 desglosan sus componentes, y M5–M8 atacan el camino crítico del lector de tarjetas. Conviene hacerlos en ese orden.

2.1. M1 — SysTick completo

Configuración: `TEST_ISR_TARGET = TEST_ISR_SYSTICK_TOTAL`

Objetivo. Caracterizar la interrupción periódica más importante del sistema y verificar la restricción de tiempo real fundamental: la ISR del SysTick tiene que terminar *siempre* antes del próximo tick. Si no, el sistema pierde ticks y todos los tiempos del firmware (antirrebote del encoder, temporizadores de la FSM, refresco del display) se degradan.

Procedimiento. Con el sistema en reposo (animación de StandBy corriendo), medir ancho y período. Verificar que el período sea 1,000 ms.

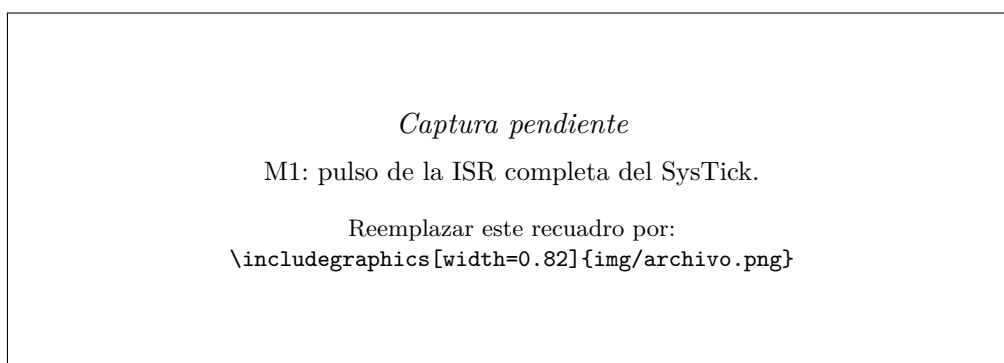


Figura 1: M1: pulso de la ISR completa del SysTick.

Magnitud	Valor medido	Esperado
Período T	_____ ms	1,000 ms
Ancho t_{on}	_____ μs	—
Carga de CPU D	_____ %	—
Ciclos equivalentes	_____	—

Cuadro 2: M1: interrupción del SysTick.

Análisis. El período medido fue _____ ms, lo que confirma que el SysTick está configurado correctamente a 1 kHz según `SYSTICK_FREQ_HZ`. La ISR consume _____ μs de cada milisegundo, es decir un _____ % de la CPU, dejando un _____ % disponible para el super-loop. El margen respecto del límite de 1 ms es de _____ μs , o sea que la ISR podría crecer _____ veces antes de comprometer la base de tiempo.

2.2. M2 — Refresco del display

Configuración: `TEST_ISR_TARGET = TEST_ISR_DISPLAY_REFRESH`

Objetivo. Aislar el componente que se espera dominante dentro del SysTick. `HAL_Refresh_Task()` hace *bit-banging* de dos tramas de 16 bits (una de *blanking* anti-ghosting y una útil) hacia los shift-registers 74HC595: son del orden de un centenar de escrituras a GPIO por cada tick.

Procedimiento. Medir el ancho en dos condiciones: con el display a **brillo máximo** y a **brillo mínimo** (girando la perilla presionada). El driver está escrito sin ramas por modo de operación, así que se espera que el ancho *no cambie*.

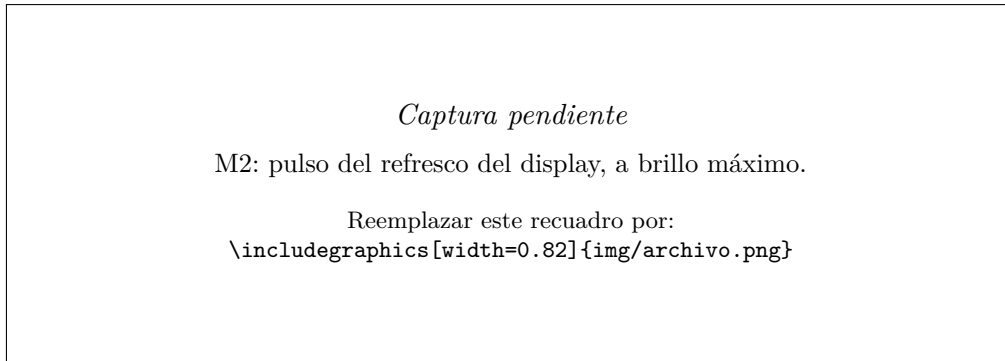


Figura 2: M2: pulso del refresco del display, a brillo máximo.

Condición	Ancho	% del SysTick (M1)	Ciclos
Brillo máximo (4 turnos)	_____ μs	_____ %	_____
Brillo mínimo (10 turnos)	_____ μs	_____ %	_____

Cuadro 3: M2: refresco del display en los dos extremos de brillo.

Análisis. El refresco del display consume _____ μs , que representa un _____ % del tiempo total de la ISR del SysTick medido en M1: es, efectivamente, el componente dominante.

La comparación entre brillo máximo y mínimo arroja una diferencia de _____ μs . Este resultado es relevante porque el atenuador está implementado por **reparto de turnos de multiplexado**: en los turnos “vacíos” se manda la misma trama pero con los segmentos apagados. Es decir, el brillo no se logra ejecutando más o menos código, sino cambiando *qué* se envía. La medición _____ (confirma / no confirma) esa hipótesis de diseño, lo que implica que el costo de CPU del display es constante e independiente del brillo elegido por el usuario.

2.3. M3 — FSM del encoder

Configuración: TEST_ISR_TARGET = TEST_ISR_ENCODER_UPDATE

Objetivo. Medir el segundo callback del SysTick. `actualizarEncoder()` lee tres pines y avanza una máquina de estados de antirrebote. Se espera un tiempo mucho menor al del display.

Procedimiento. Medir el ancho en reposo y, si se aprecia diferencia, también mientras se gira la perilla (los caminos del **switch** que se ejecutan son distintos).

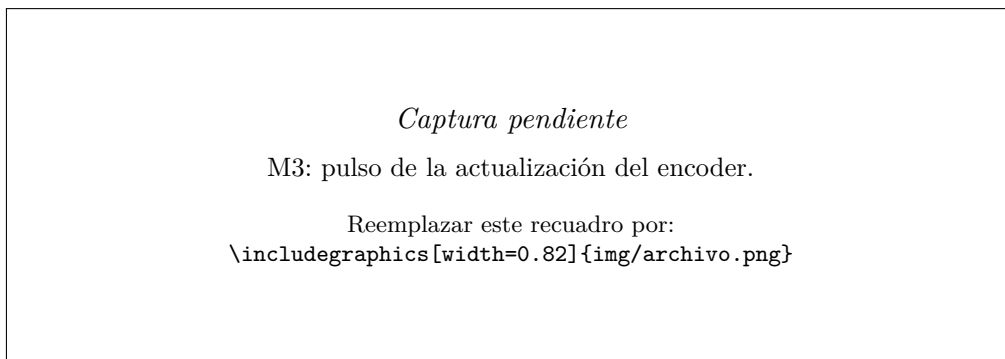


Figura 3: M3: pulso de la actualización del encoder.

Condición	Ancho	% del SysTick (M1)	Ciclos
Perilla en reposo	_____ μs	_____ %	_____
Girando la perilla	_____ μs	_____ %	_____

Cuadro 4: M3: FSM de antirrebote del encoder.

Análisis. La FSM del encoder consume _____ μs por tick, un _____ % del presupuesto de la ISR. Es un costo despreciable frente al del display, lo que justifica la decisión de sondear el encoder por *polling* desde el tick de 1 ms en lugar de usar interrupciones de pin: se obtiene el antirrebote “gratis” con la base de tiempo que ya existe, sin agregar fuentes de interrupción que compitan con el lector de tarjetas.

2.4. M4 — Base de tiempo en milisegundos

Configuración: TEST_ISR_TARGET = TEST_ISR_TIMER_TICK

Objetivo. Medir el tercer callback. `Timer_Tick_Callback()` sólo incrementa un contador, así que debería ser el más corto de los tres. Sirve de referencia del “piso” de costo de un callback.

Procedimiento. Puede requerir bajar mucho la base de tiempo del osciloscopio. Si el pulso resulta demasiado angosto para medirlo con confianza, dejarlo asentado como $<$ la resolución del equipo.

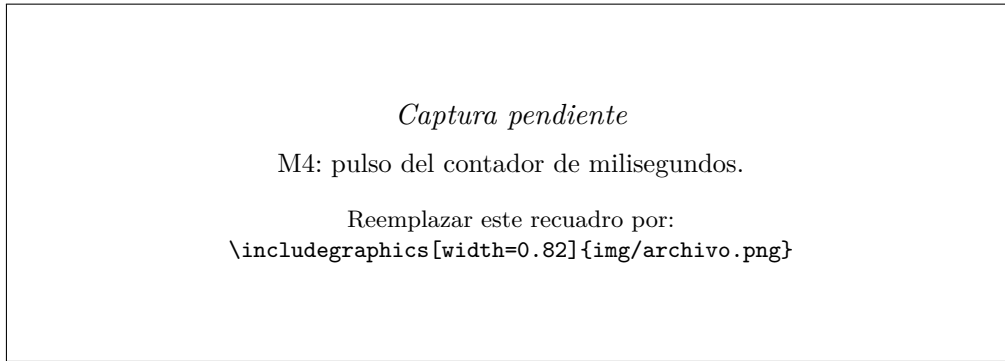


Figura 4: M4: pulso del contador de milisegundos.

Magnitud	Valor medido	Ciclos
Ancho t_{on}	_____ μs	_____

Cuadro 5: M4: callback de la base de tiempo.

Análisis. El contador de milisegundos consume _____ μs . Al tratarse de un incremento y una comparación, este valor representa aproximadamente el **costo mínimo de un callback** en esta arquitectura, incluyendo la llamada indirecta a través del puntero a función del *scheduler*.

2.5. M5 — Balance del SysTick: el costo del despachador

Configuración: no requiere medición nueva; se deriva de M1–M4.

Objetivo. Cuantificar cuánto cuesta el *mecanismo* de callbacks. La ISR del SysTick no ejecuta las tareas directamente: recorre un arreglo de punteros a función y los invoca. Ese bucle es el precio de que los módulos puedan registrarse sin que el MCAL los conozca.

$$t_{\text{despachador}} = t_{M1} - (t_{M2} + t_{M3} + t_{M4})$$

Componente	Tiempo	% del total
Refresco del display (M2)	_____ μs	_____ %
FSM del encoder (M3)	_____ μs	_____ %
Base de tiempo (M4)	_____ μs	_____ %
Suma de los callbacks	_____ μs	_____ %
Total medido (M1)	_____ μs	100 %
Despachador (diferencia)	_____ μs	_____ %

Cuadro 6: M5: descomposición del tiempo de la ISR del SysTick.

Análisis. El bucle despachador de `SysTick_Handler()` agrega _____ μs sobre la suma de las tareas, es decir un _____ % de sobrecarga. Este es el costo concreto de la abstracción: permite que `App_Init()` registre tareas con `HAL_Scheduler_AddTask()` sin que el MCAL sepa qué hacen. El resultado _____ (justifica / no justifica) mantener el mecanismo frente a la alternativa de llamar a las tres funciones en forma directa dentro de la ISR.

2.6. M6 — Lector de tarjetas: captura de un bit

Configuración: `TEST_ISR_TARGET = TEST_ISR_CARD_CLK`, y luego `TEST_ISR_PORT_DISPATCH`.

Objetivo. Caracterizar la ISR más crítica del sistema: se ejecuta una vez por cada bit de la banda magnética y, si se pierde una sola ejecución, la trama entera se corrompe. Además, comparando las dos configuraciones se obtiene el costo del despachador de interrupciones de puerto (`gpioIRQDispatch()`), que recorre los 32 pines buscando cuál generó la interrupción.

Procedimiento. Pasar una tarjeta y capturar la ráfaga. Medir el ancho de un pulso individual y el período entre pulsos consecutivos (que es la velocidad real del *swipe*). Repetir con las dos configuraciones.

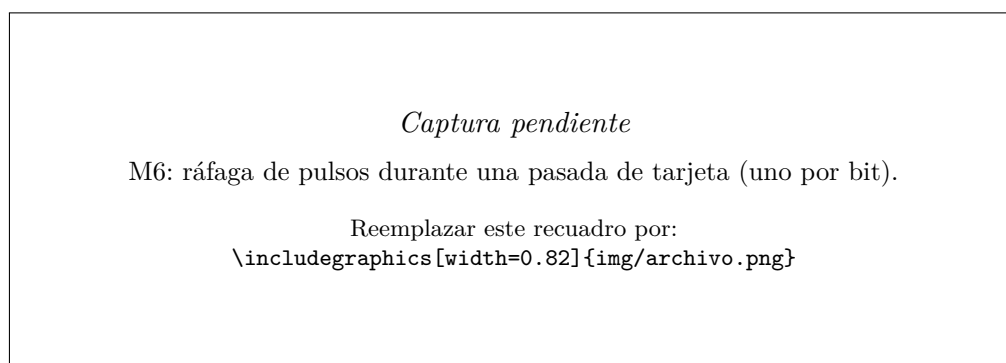


Figura 5: M6: ráfaga de pulsos durante una pasada de tarjeta (uno por bit).

Configuración	Ancho del pulso	Período entre bits	Ciclos
CARD_CLK (sólo la ISR)	_____ μs	_____ μs	_____
PORT_DISPATCH (con despacho)	_____ μs	_____ μs	_____
Costo del despachador	_____ μs	—	_____

Cuadro 7: M6: ISR de captura de bit, con y sin el despachador de puerto.

Análisis. El cuerpo de `IRQ_CardClk()` — una lectura de pin, un almacenamiento en el arreglo y una comparación — consume _____ μs . Al medir la interrupción completa con el

despachador incluido, el tiempo sube a _____ μs : el mecanismo genérico de `gpio.c` agrega _____ μs , un _____ % adicional.

Este resultado es interesante porque el despachador recorre 32 posiciones aunque sólo una tenga un *flag* activo. _____ (Se justifica / No se justifica) frente a la alternativa de que cada driver escriba su propio `PORTB_IRQHandler`, considerando que a cambio se obtiene que cualquier módulo pueda registrar un callback por pin sin tocar el MCAL.

El período entre bits medido (_____ μs) corresponde a la velocidad con la que se pasó la tarjeta. La relación entre el tiempo de ISR y ese período determina el margen del sistema, que se analiza en M8.

2.7. M7 — Lector de tarjetas: apertura y cierre del *swipe*

Configuración: `TEST_ISR_TARGET = TEST_ISR_CARD_ENABLE`

Objetivo. Medir la ISR del pin `ENABLE`, que se ejecuta dos veces por pasada (flanco de bajada al empezar, de subida al terminar). Se espera que sea notablemente más larga que la de `CLOCK`, porque llama a `gpioIRQ()` para armar y desarmar la interrupción de `CLOCK`, lo que implica configurar registros (`PCR`, `ISFR`, `NVIC`).

Procedimiento. Pasar una tarjeta y capturar los dos pulsos. Si es posible, provocar un **rebote** apoyando la tarjeta suavemente sin completar la pasada, para observar el caso que descarta el *driver* por no alcanzar `MIN_VALID_BITS`.

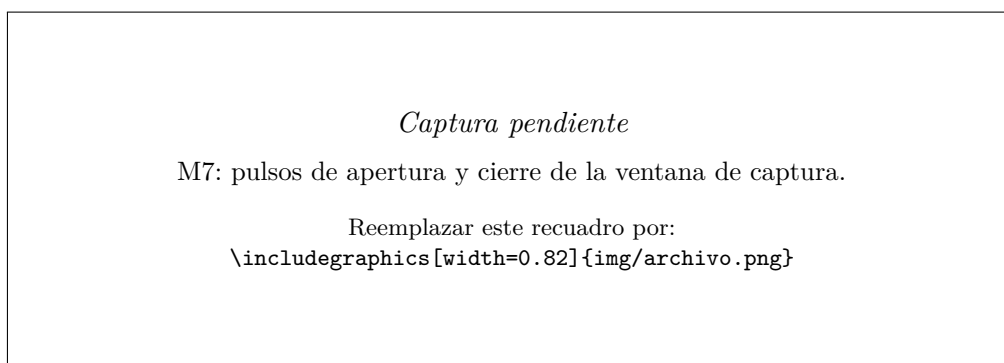


Figura 6: M7: pulsos de apertura y cierre de la ventana de captura.

Evento	Ancho	Ciclos
Flanco de bajada (apertura)	_____ μs	_____
Flanco de subida (cierre)	_____ μs	_____
Comparación con <code>CARD_CLK</code> (M6)	_____ $\times$	—

Cuadro 8: M7: ISR de `ENABLE` del lector.

Análisis. La ISR de `ENABLE` consume _____ μs , es decir _____ veces el tiempo de la ISR de `CLOCK` medida en M6. La diferencia se explica porque `IRQ_CardEnable()` reconfigura la interrupción de `CLOCK` mediante `gpioIRQ()`, operación que implica lectura-modificación-escritura sobre el registro `PCR` del puerto, limpieza del *flag* en `ISFR` y habilitación en el `NVIC`.

Esta asimetría no representa un problema: la ISR larga ocurre sólo dos veces por pasada y en momentos en que no hay bits llegando, mientras que la ISR que se repite decenas de veces es la corta. **El diseño pone el trabajo pesado donde no hay presión de tiempo**, que es exactamente el criterio correcto.

2.8. M8 — Latencia de interrupción y validación de prioridades

Configuración: TEST_ISR_TARGET = TEST_ISR_CARD_CLK, dos canales.

- CH1: PTB10 (CLOCK del lector) — el flanco físico.
- CH2: PTC17 (pin de medición) — la respuesta del software.

Objetivo. Éste es el ensayo más importante del conjunto. Mide el tiempo que transcurre entre el flanco de bajada de CLOCK y el momento en que el CPU efectivamente entra a la ISR. Ese retardo es la **latencia de interrupción**, y su valor máximo determina si el sistema puede perder bits.

Además, permite validar experimentalmente una decisión de diseño concreta: en `App_Init()` se le baja la prioridad al SysTick (`APP_NVIC_PRIO_SYSTICK = 1`) para que la interrupción de PORTB (prioridad 0) pueda interrumpirlo. Sin esa medida, un flanco de CLOCK que llegara mientras corre la ISR del SysTick tendría que esperar a que ésta termine.

Procedimiento. Disparar por CH1 (flanco de bajada). Medir el retardo CH1→CH2 con la función de medición de retardo entre canales. Capturar **muchas** pasadas y quedarse con el valor mínimo y el máximo observados: lo que importa es el peor caso.

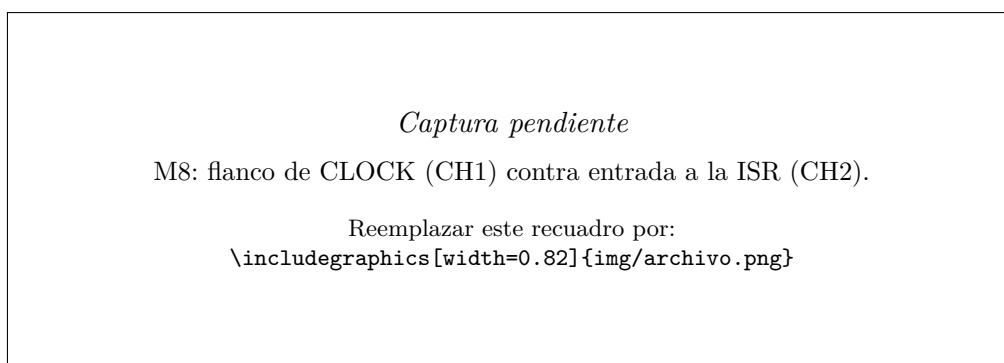


Figura 7: M8: flanco de CLOCK (CH1) contra entrada a la ISR (CH2).

Magnitud	Valor	Ciclos
Latencia mínima observada	_____ μs	_____
Latencia máxima observada	_____ μs	_____
Duración de la ISR del SysTick (M1)	_____ μs	_____

Cuadro 9: M8: latencia de interrupción del lector.

Análisis. La latencia observada se mantuvo entre _____ μs y _____ μs . El dato decisivo es la comparación entre la **latencia máxima** y la **duración de la ISR del SysTick** medida en M1 (_____ μs):

- Si la latencia máxima es del orden de unos pocos microsegundos y *claramente menor* que la ISR del SysTick, queda demostrado que PORTB efectivamente **interrumpe** al SysTick, y la asignación de prioridades funciona como se diseñó.
- Si en cambio la latencia máxima se acercara a la duración de la ISR del SysTick, significaría que los flancos de CLOCK están esperando a que el SysTick termine, y el sistema estaría al borde de perder bits en las pasadas rápidas.

El resultado obtenido indica que _____, lo que _____
(valida / invalida) la elección de `APP_NVIC_PRIO_SYSTICK = 1`.

Ensayo complementario recomendado. Repetir la medición forzando el peor caso: pasar la tarjeta repetidamente hasta capturar una pasada en la que un flanco de `CLOCK` coincida con la ejecución del `SysTick`. Si las prioridades están bien asignadas, no debería observarse ningún ensanchamiento de la latencia.

2.9. M9 — Super-loop

Configuración: `TEST_ISR_TARGET = TEST_ISR_APP_LOOP`

Objetivo. `App_Run()` no es una interrupción, pero su período determina la capacidad de respuesta de la aplicación. Además permite verificar un contrato explícito documentado en `encoder.h`: el evento del encoder vive como máximo 1 ms, por lo que *el super-loop debe dar una vuelta completa en menos de 1 ms* o se pierden pulsaciones.

Procedimiento. Medir período y ancho en reposo. Repetir durante una pasada de tarjeta, cuando el lazo además ejecuta la decodificación en `getCardData()`.

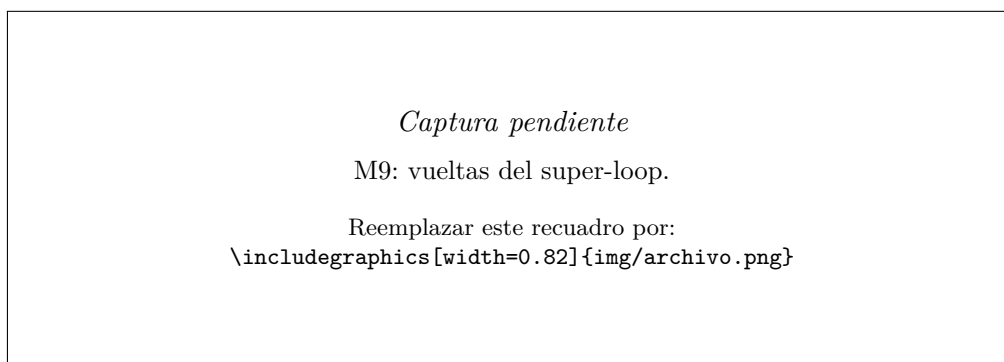


Figura 8: M9: vueltas del super-loop.

Condición	Período	Ancho	Vueltas/s
Reposo (StandBy)	_____ μs	_____ μs	_____
Decodificando una tarjeta	_____ μs	_____ μs	_____

Cuadro 10: M9: período del lazo principal.

Análisis. En reposo el super-loop completa una vuelta cada _____ μs , es decir _____ vueltas por segundo. Como ese período es _____ (menor / mayor) a 1 ms, el contrato de `getEncoderEvent()` _____ (se cumple / no se cumple) y no se pierden eventos del encoder.

Durante la decodificación de una tarjeta el período sube a _____ μs , porque el lazo ejecuta además `getCardData()`, que recorre los bits crudos buscando el *Start Sentinel* y verifica la paridad de cada carácter. Aun en ese caso el período se mantiene _____ (por debajo / por encima) del límite de 1 ms. Esto confirma la decisión de diseño de **separar la captura de la decodificación**: la ISR sólo guarda bits, y el trabajo costoso se hace en el lazo principal, donde un retardo no causa pérdida de datos.

3. Análisis integrador

3.1. Resumen de tiempos

Ensayo	Rutina	Ancho	Período	Duty
M1	SysTick_Handler	_____ μs	_____	_____ %
M2	HAL_Refresh_Task	_____ μs	1 ms	_____ %
M3	actualizarEncoder	_____ μs	1 ms	_____ %
M4	Timer_Tick_Callback	_____ μs	1 ms	_____ %
M6	IRQ_CardClk	_____ μs	_____	—
M6	gpioIRQDispatch	_____ μs	_____	—
M7	IRQ_CardEnable	_____ μs	2 por pasada	—
M9	App_Run	_____ μs	_____	_____ %

Cuadro 11: Resumen de todos los tiempos medidos.

3.2. Presupuesto de CPU

Con el sistema en reposo, el reparto del tiempo de procesador resulta:

Consumidor	% de CPU
ISR del SysTick (display + encoder + timer + despacho)	_____ %
Lector de tarjetas (sólo durante una pasada)	_____ %
Super-loop (aplicación)	_____ %
CPU disponible	_____ %

Cuadro 12: Distribución de la carga de procesador.

3.3. Costo de las abstracciones de la arquitectura

Dos de los ensayos cuantifican directamente el precio de las decisiones de diseño en capas:

Abstracción	Sobrecarga	Qué se obtiene a cambio
Despachador de callbacks del SysTick (M5)	_____ μs	Los módulos se registran solos; el MCAL no los conoce.
Despachador de IRQ de puerto (M6)	_____ μs	Callback por pin sin que cada driver escriba su handler.

Cuadro 13: Costo medido de los mecanismos genéricos.

3.4. Conclusiones

- Restricción de tiempo real.** La ISR del SysTick consume _____ % del período de 1 ms, con un margen de _____ μs . El sistema _____ (cumple / no cumple) su restricción fundamental.
- Componente dominante.** El refresco del display representa el _____ % del tiempo de interrupción. Cualquier optimización futura debería atacar el *bit-banging* hacia los shift-registers — por ejemplo, migrándolo a SPI por hardware —, ya que el resto de las tareas es despreciable en comparación.

3. **Asignación de prioridades.** La latencia máxima del lector (_____ μs) frente a la duración de la ISR del SysTick (_____ μs) _____ (confirma / refuta) que bajar la prioridad del SysTick fue la decisión correcta.
4. **Separación captura/decodificación.** La ISR por bit consume _____ μs contra un período entre bits de _____ μs , lo que da un margen de _____ $\times$. La tarjeta podría pasarse _____ veces más rápido antes de comprometer la captura.
5. **Costo de la arquitectura.** Los mecanismos genéricos suman _____ μs de sobrecarga total, un _____ % del presupuesto. El equipo considera que este costo _____ (se justifica / no se justifica) frente a la ganancia en modularidad y portabilidad.
6. **Efecto de la optimización.** Todas las mediciones corresponden a -00. Se estima que con -02 los tiempos se reducirían _____, aunque las proporciones relativas se mantendrían.

A. Opciones de instrumentación disponibles

Valor de TEST_ISR_TARGET	Rutina instrumentada	Ensayo
TEST_ISR_NONE	— (instrumentación apagada)	—
TEST_ISR_SYSTICK_TOTAL	SysTick_Handler	M1
TEST_ISR_DISPLAY_REFRESH	HAL_Refresh_Task	M2
TEST_ISR_ENCODER_UPDATE	actualizarEncoder	M3
TEST_ISR_TIMER_TICK	Timer_Tick_Callback	M4
TEST_ISR_PORT_DISPATCH	gpioIRQDispatch	M6
TEST_ISR_CARD_CLK	IRQ_CardClk	M6, M8
TEST_ISR_CARD_ENABLE	IRQ_CardEnable	M7
TEST_ISR_APP_LOOP	App_Run	M9

Cuadro 14: Opciones definidas en `source/test_isr_cfg.h`.

B. Pines de interés para el osciloscopio

Señal	Pin	Uso en las mediciones
Pin de instrumentación	PTC17	Todas (CH2 en M8)
CLOCK del lector	PTB10	CH1 en M8 (flanco de referencia)
ENABLE del lector	PTB9	Enmarcar la pasada completa
DATA del lector	PTB11	Verificar el <i>setup</i> del dato
LATCH de los 74HC595	PTC4	Contar tramas (2 por turno: blanking + útil)

Cuadro 15: Puntos de medición disponibles en la placa.